

BlackPebble — Replit Build Prompt (Phase 1)

Professional Solana Paper Trading & Analytics Platform

PROJECT OVERVIEW

BlackPebble is a professional Solana utility platform. Phase 1 focuses exclusively on building a production-ready paper trading simulator with real market data, including pre-migration PumpFun bonding curve tokens.

This is an interactive TOOL — not an info site, not a token project page, not a landing page. Users open it daily to practice trading, track markets, and sharpen their skills risk-free.

What BlackPebble IS:

- A paper trading platform using 100% real Solana market data
- A token discovery and market analytics tool
- A professional utility platform that will expand with additional tools over time

What BlackPebble is NOT:

- A token project website
- A memecoin info page
- A landing page with walls of text
- Anything related to a specific token launch

No references anywhere on the site to: token launches, contract addresses, airdrops, tokenomics, distributions, or fund management. BlackPebble is a standalone utility platform.

TECH STACK

- Frontend: React + Vite
- Backend: Express.js (Node)
- Database: SQLite (via better-sqlite3)
- Wallet: @solana/wallet-adapter (Phantom + Solflare)
- Charts: DexScreener embed + Chart.js

- Live Data: PumpPortal WebSocket (pre-migration) + Jupiter Price API + DexScreener API (post-migration)
 - Primary Solana Infrastructure: Helius (API key already in Replit Secrets as HELIUS_API_KEY)
 - Scheduling: node-cron
 - Hosting: Replit
-

HELIUS AS PRIMARY DATA LAYER

Helius is the primary Solana infrastructure provider. Use it for:

- RPC requests (replace all public RPC calls with Helius RPC)
- Token metadata (names, symbols, logos, decimals)
- Wallet balances and token holdings
- Transaction history

Helius RPC endpoint: `https://mainnet.helius-rpc.com/?api-key=${HELIUS_API_KEY}`

Helius DAS API (for metadata): `https://mainnet.helius-rpc.com/?api-key=${HELIUS_API_KEY}`

Use public endpoints (mainnet-beta.solana.com) ONLY as fallback if Helius is unavailable.

ARCHITECTURE PRINCIPLES

Modular Design:

- Backend organized as independent service modules (trading, markets, analytics, etc.)
- Each module has its own routes, controllers, and database tables
- New utility modules (wallet cleanup, burns, etc.) can be added later without refactoring
- Navigation designed to accommodate future tool sections

Phase Separation:

- Phase 1 (NOW): Trading Desk, Markets, Portfolio, Wallet Connection — fully production-ready
- Phase 2 (LATER): Leaderboards, Competitions — uses data collected in Phase 1
- Phase 3 (LATER): Vault, Rewards, Additional Utilities — only after core is stable

Build all database schemas now (including for Phase 2/3), but only implement Phase 1 UI and logic. Phase 2/3 pages show "Coming Soon" states.

DESIGN SYSTEM

Colors:

Plain Text

```
--bg-primary: #0a0a0a
--bg-secondary: #111111
--bg-card: #141414
--bg-elevated: #1a1a1a
--border: #1f1f1f
--border-hover: #2a2a2a
--text-primary: #ffffff
--text-secondary: #a0a0a0
--text-muted: #666666
--accent: #c9a96e (gold – used sparingly for key actions only)
--accent-dim: #8a7040
--positive: #22c55e
--negative: #ef4444
--info: #3b82f6
```

Typography:

- All text: Inter (clean, modern sans-serif)
- Numbers/prices/data: JetBrains Mono (monospace)
- Font sizes: 14px base, 12px data tables, 24-32px key stats

Layout:

- Dashboard-style with panels/cards
- Dense but readable (trading platform, not blog)
- Sidebar navigation on desktop
- Full-screen utilization on Trading Desk (no max-width container)
- Smooth transitions, skeleton loading states
- No clutter, no decorative elements, no unnecessary whitespace

Components:

- Cards: #141414 bg, 1px #1f1f1f border, 8px radius
- Buttons: primary = gold fill, secondary = outlined, ghost = text only
- Inputs: #111 bg, subtle border, gold focus ring
- Tables: compact rows, subtle alternating bg, hover highlight

- Tabs: underline style with gold active indicator

NAVIGATION

Desktop — Left Sidebar (60px collapsed, 200px on hover):

Plain Text

[BP Logo]

Trading Desk	← Phase 1 (active)
Markets	← Phase 1 (active)
Portfolio	← Phase 1 (active)
Leaderboard	← Phase 2 (shows "Coming Soon")
Utilities	← Phase 3 (shows "Coming Soon")

[Connect Wallet]

Use clean line icons (Lucide or similar). No emojis.

Mobile — Bottom Tab Bar:

Plain Text

Trading | Markets | Portfolio | More

"More" opens a sheet with: Leaderboard (coming soon), Utilities (coming soon), Settings.

Top Bar (persistent on all pages):

- Left: "BlackPebble" logo/wordmark
- Center: Global token search bar (always accessible)
- Right: Wallet status (connected: truncated address + paper balance | disconnected: "Connect" button)

WALLET CONNECTION (PRIORITY #1 — MUST WORK)

Fix the current broken implementation:

JSX

```
// At app root level:  
import { ConnectionProvider, WalletProvider } from '@solana/wallet-adapter-react'  
import { WalletModalProvider } from '@solana/wallet-adapter-react-ui';
```

```
import { PhantomWalletAdapter, SolflareWalletAdapter } from '@solana/wallet-ada
import '@solana/wallet-adapter-react-ui/styles.css';

const wallets = [new PhantomWalletAdapter(), new SolflareWalletAdapter()];
const endpoint = `https://mainnet.helius-rpc.com/?api-key=${import.meta.env.VIT

function App( ) {
  return (
    <ConnectionProvider endpoint={endpoint}>
      <WalletProvider wallets={wallets} autoConnect>
        <WalletModalProvider>
          <Router>{ /* routes */ }</Router>
        </WalletModalProvider>
      </WalletProvider>
    </ConnectionProvider>
  );
}
```

Important: Set `VITE_HELIUS_RPC_URL` in Replit Secrets only if needed for frontend wallet adapter connection. Do not expose the raw Helius API key anywhere client-side beyond the RPC URL. All sensitive Helius calls must go through backend routes.

The site should be BROWSABLE without connecting (users can view markets, search tokens, see charts). But trading and portfolio features require wallet connection. Show a clean "Connect Wallet to Start Trading" prompt on the trade panel if not connected.

PAGE 1: TRADING DESK (Route: / — HOMEPAGE)

This is the primary product. Users land here.

Desktop Layout (Full-Screen Terminal):

Plain Text

	[Global Search Bar]	Balance: 100 SOL [Wallet]
SIDEBAR	CHART AREA (DexScreener embed for migrated or Chart.js for bonding curve)	TRADE PANEL
	Avg Entry: \$0.00002 P&L: +2.5 SOL (+50%)	[BUY] [SELL]
		Amount: [____] SOL [1][5][10][25%][MAX]
		≈ 250,000 tokens


```
style={{ width: '100%', height: '100%', border: 'none' }}
/>
```

Pre-migration (bonding curve) tokens:

- Custom Chart.js line chart from PumpPortal trade data
- Plot price over last 100 trades
- Show bonding curve progress bar: "Bonding: 67% "

Overlays (if user holds token):

- Avg entry price label (gold, bottom-left)
- Live P&L badge (green/red, top-right)

Token Info Bar:

| Name & Ticker | Price | Market Cap | 24h Volume | 24h Change | Liquidity |

For bonding curve tokens also: | Bonding Progress | Virtual SOL Reserves |

Trade Panel:

BUY tab:

- SOL amount input
- Quick buttons: 1 | 5 | 10 | 25% | 50% | MAX
- Shows: estimated tokens, price per token
- "Buy" button (gold)
- Disabled if not connected or price unavailable

SELL tab:

- Token amount input (only if holding)
- Quick buttons: 25% | 50% | 75% | ALL
- Shows: estimated SOL received, P&L for this trade
- "Sell" button (red)

After trade: Success toast (3s auto-dismiss), balance updates instantly, position list updates.

Live Trade Feed (right panel desktop, collapsible on mobile):

- Real-time trades from PumpPortal WebSocket
- Shows: time, wallet (4...4), buy/sell, SOL amount

- Buys green, sells red
- Scrolling, most recent top, max 50 visible
- Makes the platform feel alive

Open Positions Tab:

| Token | Entry | Current | Qty | Value (SOL) | P&L | P&L % | [Sell] |

- Prices update every 15 seconds
- Click token → loads chart
- Click Sell → opens sell panel for that token

Trade History Tab:

| Time | Token | Side | Amount | Price | P&L |

- Most recent first, paginated (20/page)

Watchlist Tab:

- Star tokens from search to save here
 - Shows: token, price, 24h%, MC
 - Click → loads into trading view
 - Max 50 items
-

PAGE 2: MARKETS (Route: /markets)

Token discovery — trending, new launches, migrations.

Tabs: Trending | New Launches | Recently Migrated | Top Gainers

Trending:

- Top tokens by volume from DexScreener
- Card grid: logo, name, ticker, price, 24h%, MC, mini sparkline
- Click → loads in Trading Desk

New Launches:

- Live feed from PumpPortal (subscribeNewToken)
- Shows: name, ticker, time created, creator (truncated), bonding %, initial activity
- Real-time — new tokens appear at top as they launch

- Click → loads in Trading Desk

Recently Migrated:

- From PumpPortal (subscribeMigration)
- Shows: name, ticker, migration time, current MC, post-migration volume

Top Gainers:

- Sorted by price change
- Filter: 1h | 6h | 24h
- From DexScreener data

PAGE 3: PORTFOLIO (Route: /portfolio)

Personal dashboard (requires wallet connection).

Overview:

Plain Text

Paper Balance	Portfolio Val	Total Value	All-Time P&L
45.2 SOL	87.3 SOL	132.5 SOL	+32.5 (+32%)

Performance Chart:

- Chart.js line chart: portfolio value over time (daily snapshots)
- Gold line on dark background
- Timeframe selector: 7d | 30d | All

Stats Grid:

| Total Trades | Win Rate | Best Trade | Worst Trade | Avg Hold | Current Streak |

Open Positions (full view):

Same as Trading Desk but expanded with more detail.

Trade History (full view):

Paginated, sortable by date/token/P&L.

PAGE 4: LEADERBOARD (Route: /leaderboard) — PHASE 2 PLACEHOLDER

Show a clean "Coming Soon" state:

- BlackPebble logo
- "Leaderboard — Coming Soon"
- "Compete against other traders. Rankings, competitions, and rewards launching soon."
- Subtle animated background or gradient
- No broken UI, no empty tables

The database schema for leaderboard data should still be created (see Database section). Just don't build the full UI yet.

PAGE 5: UTILITIES (Route: /utilities) — PHASE 3 PLACEHOLDER

Show a clean "Coming Soon" state:

- "Solana Utilities — Coming Soon"
- "Wallet cleanup, token burns, rent recovery, and more."
- Brief list of planned tools (text only, no functionality)

Architecture note: the navigation and routing should already support adding utility sub-pages later (e.g., /utilities/cleanup, /utilities/burn) without restructuring.

PUMPPORTAL WEBSOCKET SERVICE (Backend)

Persistent WebSocket connection for live PumpFun data:

JavaScript

```
const WebSocket = require('ws');

class PumpPortalService {
  constructor() {
    this.ws = null;
    this.tokenPrices = new Map(); // mint -> { price, marketCapSol, bondingProg
    this.recentTrades = new Map(); // mint -> array of last 100 trades
    this.newTokens = []; // last 100 new launches
    this.migrations = []; // last 50 migrations
  }
}
```

```

    this.connect();
  }

  connect() {
    this.ws = new WebSocket('wss://pumpportal.fun/api/data');

    this.ws.on('open', () => {
      console.log('[PumpPortal] Connected');
      this.subscribeNewTokens();
      this.subscribeMigrations();
      this.resubscribeTokens();
    });

    this.ws.on('message', (data) => {
      try {
        const msg = JSON.parse(data);
        this.handleMessage(msg);
      } catch (e) {}
    });

    this.ws.on('close', () => {
      console.log('[PumpPortal] Disconnected, reconnecting in 5s');
      setTimeout(() => this.connect(), 5000);
    });

    this.ws.on('error', (err) => {
      console.error('[PumpPortal] Error:', err.message);
    });
  }

  subscribeToken(mint) {
    if (this.ws?.readyState === WebSocket.OPEN) {
      this.ws.send(JSON.stringify({ method: "subscribeTokenTrade", keys: [mint] }));
    }
    if (!this.recentTrades.has(mint)) {
      this.recentTrades.set(mint, []);
    }
  }

  unsubscribeToken(mint) {
    if (this.ws?.readyState === WebSocket.OPEN) {
      this.ws.send(JSON.stringify({ method: "unsubscribeTokenTrade", keys: [mint] }));
    }
  }

  subscribeNewTokens() {
    if (this.ws?.readyState === WebSocket.OPEN) {
      this.ws.send(JSON.stringify({ method: "subscribeNewToken" }));
    }
  }

```

```

    }
  }

  subscribeMigrations() {
    if (this.ws?.readyState === WebSocket.OPEN) {
      this.ws.send(JSON.stringify({ method: "subscribeMigration" }));
    }
  }

  handleMessage(msg) {
    // New token creation
    if (msg.txType === 'create' || msg.signature && msg.name && !msg.traderPubl
      this.newTokens.unshift({ ...msg, timestamp: Date.now() });
      if (this.newTokens.length > 100) this.newTokens.pop();
      return;
    }

    // Migration event
    if (msg.txType === 'migration' || msg.pool) {
      this.migrations.unshift({ ...msg, timestamp: Date.now() });
      if (this.migrations.length > 50) this.migrations.pop();
      return;
    }

    // Trade event
    if (msg.mint) {
      const trades = this.recentTrades.get(msg.mint) || [];
      trades.unshift({
        timestamp: Date.now(),
        wallet: msg.traderPublicKey,
        side: msg.txType,
        solAmount: msg.solAmount,
        tokenAmount: msg.tokenAmount,
        vSolInBondingCurve: msg.vSolInBondingCurve,
        vTokensInBondingCurve: msg.vTokensInBondingCurve,
        marketCapSol: msg.marketCapSol
      });
      if (trades.length > 100) trades.pop();
      this.recentTrades.set(msg.mint, trades);
    }

    // Update price
    if (msg.vSolInBondingCurve && msg.vTokensInBondingCurve) {
      const price = msg.vSolInBondingCurve / msg.vTokensInBondingCurve;
      // Bonding curve starts at ~30 SOL virtual, migrates at ~85 SOL
      const bondingProgress = Math.min(Math.max((msg.vSolInBondingCurve - 30
      this.tokenPrices.set(msg.mint, {
        price,
        marketCapSol: msg.marketCapSol,

```

```

        bondingProgress,
        lastUpdate: Date.now()
    });
    }
}

getPrice(mint) { return this.tokenPrices.get(mint) || null; }
getTrades(mint) { return this.recentTrades.get(mint) || []; }
getNewTokens() { return this.newTokens; }
getMigrations() { return this.migrations; }

resubscribeTokens() {
    const mints = Array.from(this.recentTrades.keys());
    if (mints.length > 0 && this.ws?.readyState === WebSocket.OPEN) {
        this.ws.send(JSON.stringify({ method: "subscribeTokenTrade", keys: mints }));
    }
}
}

module.exports = new PumpPortalService();

```

Frontend WebSocket Relay:

Backend also runs a WebSocket server that relays PumpPortal data to connected frontend clients:

JavaScript

```

const { WebSocketServer } = require('ws');

// Attach to Express server's HTTP instance
const wss = new WebSocketServer({ noServer: true });

wss.on('connection', (ws) => {
    ws.subscribedTokens = new Set();

    ws.on('message', (data) => {
        const msg = JSON.parse(data);
        if (msg.type === 'subscribe' && msg.mint) {
            ws.subscribedTokens.add(msg.mint);
            pumpPortalService.subscribeToken(msg.mint);
        }
        if (msg.type === 'unsubscribe' && msg.mint) {
            ws.subscribedTokens.delete(msg.mint);
        }
    });
});

```

```
});

// Broadcast trade to subscribed clients
function broadcastTrade(mint, tradeData) {
  wss.clients.forEach(client => {
    if (client.readyState === WebSocket.OPEN && client.subscribedTokens.has(mint)) {
      client.send(JSON.stringify({ type: 'trade', mint, data: tradeData }));
    }
  });
}
```

PRICE FETCHING STRATEGY

Bonding curve tokens (pre-migration):

- Primary: PumpPortal WebSocket trade events (real-time)
- Price = $vSolInBondingCurve / vTokensInBondingCurve$

Migrated tokens (on Raydium/Jupiter/PumpSwap):

- Primary: Jupiter Price API: GET <https://price.jup.ag/v6/price?ids={mint}>
- Fallback: DexScreener: GET <https://api.dexscreener.com/latest/dex/tokens/{mint}>

SOL/USD (for USD display):

- Jupiter: GET <https://price.jup.ag/v6/price?ids=So11111111111111111111111111111111112>
- Cache 30 seconds

Token metadata (name, symbol, logo, decimals):

- Primary: Helius DAS API
- Fallback: DexScreener search results (include metadata)

All external API calls through backend (CORS, caching, rate limiting).

BACKEND API ENDPOINTS

Plain Text

Account

POST	/api/account/create	– Create paper account (100 SOL start)
GET	/api/account/:wallet	– Get account info

Trading

POST	/api/trade/search	– Search tokens (proxy DexScreener + Pump)
------	-------------------	--

GET	/api/trade/token/:mint	– Token info + price + bonding status
POST	/api/trade/execute	– Execute paper trade (buy/sell)
GET	/api/trade/positions/:wallet	– Open positions with live prices
GET	/api/trade/history/:wallet	– Trade history (paginated)
POST	/api/trade/watchlist/add	– Add to watchlist
POST	/api/trade/watchlist/remove	– Remove from watchlist
GET	/api/trade/watchlist/:wallet	– Get watchlist
# Live Data		
GET	/api/live/trades/:mint	– Recent trades (from PumpPortal cache)
GET	/api/live/new-tokens	– Latest PumpFun launches
GET	/api/live/migrations	– Recent migrations
WS	/ws	– WebSocket relay for frontend
# Markets		
GET	/api/markets/trending	– Trending (DexScreener)
GET	/api/markets/gainers?tf=1h 6h 24h	– Top gainers
GET	/api/markets/volume	– Top volume
# Portfolio		
GET	/api/portfolio/:wallet	– Full overview
GET	/api/portfolio/chart/:wallet	– Value history for chart
GET	/api/portfolio/stats/:wallet	– Performance stats

DATABASE SCHEMA

Create ALL tables now (Phase 2/3 tables will be empty but ready):

SQL

```
-- PHASE 1: Core Trading
CREATE TABLE accounts (
  wallet TEXT PRIMARY KEY,
  paper_balance REAL DEFAULT 100.0,
  total_trades INTEGER DEFAULT 0,
  winning_trades INTEGER DEFAULT 0,
  total_pnl REAL DEFAULT 0.0,
  best_trade REAL DEFAULT 0.0,
  worst_trade REAL DEFAULT 0.0,
  current_streak INTEGER DEFAULT 0,
  created_at INTEGER DEFAULT (unixepoch()),
  last_active INTEGER DEFAULT (unixepoch())
);

CREATE TABLE positions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
wallet TEXT NOT NULL,  
token_mint TEXT NOT NULL,  
token_name TEXT,  
token_symbol TEXT,  
total_tokens REAL NOT NULL,  
total_sol_spent REAL NOT NULL,  
avg_entry_price REAL NOT NULL,  
opened_at INTEGER DEFAULT (unixepoch()),  
UNIQUE(wallet, token_mint)  
);
```

```
CREATE TABLE trades (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  wallet TEXT NOT NULL,  
  token_mint TEXT NOT NULL,  
  token_name TEXT,  
  token_symbol TEXT,  
  side TEXT NOT NULL, -- 'buy' or 'sell'  
  sol_amount REAL NOT NULL,  
  token_amount REAL NOT NULL,  
  price REAL NOT NULL,  
  pnl REAL, -- only for sells  
  executed_at INTEGER DEFAULT (unixepoch())  
);
```

```
CREATE TABLE watchlist (  
  wallet TEXT NOT NULL,  
  token_mint TEXT NOT NULL,  
  token_name TEXT,  
  token_symbol TEXT,  
  added_at INTEGER DEFAULT (unixepoch()),  
  PRIMARY KEY(wallet, token_mint)  
);
```

-- PHASE 1: Analytics (collect data now, build dashboards later)

```
CREATE TABLE token_views (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  wallet TEXT,  
  token_mint TEXT NOT NULL,  
  viewed_at INTEGER DEFAULT (unixepoch())  
);
```

```
CREATE TABLE search_activity (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  wallet TEXT,  
  query TEXT NOT NULL,  
  results_count INTEGER,  
  searched_at INTEGER DEFAULT (unixepoch())
```



```

);

-- PHASE 2: Leaderboard & Competitions (schema ready, not populated yet)
CREATE TABLE leaderboard_snapshots (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  wallet TEXT NOT NULL,
  period TEXT NOT NULL, -- 'all', 'month', 'week'
  period_start INTEGER,
  total_pnl REAL,
  roi_percent REAL,
  win_rate REAL,
  trade_count INTEGER,
  rank INTEGER,
  snapshot_at INTEGER DEFAULT (unixepoch())
);

CREATE TABLE competitions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  period_type TEXT NOT NULL, -- 'weekly', 'monthly'
  start_at INTEGER NOT NULL,
  end_at INTEGER NOT NULL,
  status TEXT DEFAULT 'active', -- 'active', 'completed'
  created_at INTEGER DEFAULT (unixepoch())
);

CREATE TABLE competition_results (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  competition_id INTEGER NOT NULL,
  wallet TEXT NOT NULL,
  pnl REAL,
  rank INTEGER,
  FOREIGN KEY(competition_id) REFERENCES competitions(id)
);

-- PHASE 3: Participation metrics (store now for future rewards)
CREATE TABLE participation_metrics (
  wallet TEXT NOT NULL,
  date TEXT NOT NULL, -- YYYY-MM-DD
  trades_today INTEGER DEFAULT 0,
  points_today INTEGER DEFAULT 0,
  PRIMARY KEY(wallet, date)
);

-- PHASE 3: Future utility module tracking
CREATE TABLE utility_usage (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  wallet TEXT NOT NULL,
  utility_type TEXT NOT NULL, -- 'cleanup', 'burn', 'close_account', etc.

```

```

details TEXT, -- JSON blob
executed_at INTEGER DEFAULT (unixepoch())
);

-- Indexes
CREATE INDEX idx_trades_wallet ON trades(wallet);
CREATE INDEX idx_trades_executed ON trades(executed_at);
CREATE INDEX idx_positions_wallet ON positions(wallet);
CREATE INDEX idx_token_views_mint ON token_views(token_mint);
CREATE INDEX idx_search_activity_query ON search_activity(query);
CREATE INDEX idx_participation_wallet ON participation_metrics(wallet);

```

TRADE EXECUTION LOGIC

JavaScript

```

async function executeTrade(wallet, tokenMint, tokenName, tokenSymbol, side, amount) {
  const account = db.prepare('SELECT * FROM accounts WHERE wallet = ?').get(wallet);
  if (!account) throw new Error('Account not found');

  // Get current price (try PumpPortal first, then Jupiter, then DexScreener)
  let priceInSol = null;
  const pumpData = pumpPortalService.getPrice(tokenMint);
  if (pumpData) {
    priceInSol = pumpData.price;
  } else {
    priceInSol = await getJupiterPriceInSol(tokenMint);
  }
  if (!priceInSol) {
    priceInSol = await getDexScreenerPriceInSol(tokenMint);
  }
  if (!priceInSol) throw new Error('Price unavailable');

  const SLIPPAGE = 0.99; // 1% simulated slippage

  if (side === 'buy') {
    if (account.paper_balance < amount) throw new Error('Insufficient balance');
    if (amount < 0.1) throw new Error('Minimum trade is 0.1 SOL');

    // Check max positions
    const posCount = db.prepare('SELECT COUNT(*) as c FROM positions WHERE wallet = ?').get(wallet);
    if (posCount.c >= 20) throw new Error('Maximum 20 open positions');

    const tokensReceived = (amount / priceInSol) * SLIPPAGE;

    // Update balance
  }
}

```

```

db.prepare('UPDATE accounts SET paper_balance = paper_balance - ?, total_trades = total_trades + 1').run(amount, wallet);

// Create or update position
const existing = db.prepare('SELECT * FROM positions WHERE wallet = ? AND token_mint = ?').run(wallet, tokenMint);
if (existing) {
  const newTotalTokens = existing.total_tokens + tokensReceived;
  const newTotalSol = existing.total_sol_spent + amount;
  const newAvgEntry = newTotalSol / newTotalTokens * SLIPPAGE;
  db.prepare('UPDATE positions SET total_tokens = ?, total_sol_spent = ?, avg_entry = ?').run(newTotalTokens, newTotalSol, priceInSol, wallet, tokenMint);
} else {
  db.prepare('INSERT INTO positions (wallet, token_mint, token_name, token_symbol, total_tokens, total_sol_spent, avg_entry)').run(wallet, tokenMint, tokenName, tokenSymbol, tokensReceived, amount, newAvgEntry);
}

// Log trade
db.prepare('INSERT INTO trades (wallet, token_mint, token_name, token_symbol, amount, tokens_received)').run(wallet, tokenMint, tokenName, tokenSymbol, 'buy', amount, tokensReceived);

// Track participation
const today = new Date().toISOString().split('T')[0];
db.prepare('INSERT INTO participation_metrics (wallet, date, trades_today, total_tokens, total_sol_spent)').run(wallet, today, tradesToday, totalTokens, totalSolSpent);

// Track analytics
db.prepare('INSERT INTO token_views (wallet, token_mint) VALUES (?, ?)').run(wallet, tokenMint);

} else if (side === 'sell') {
  const position = db.prepare('SELECT * FROM positions WHERE wallet = ? AND token_mint = ?').run(wallet, tokenMint);
  if (!position) throw new Error('No position found');
  if (position.total_tokens < amount) throw new Error('Insufficient tokens');

  const solReceived = (amount * priceInSol) * SLIPPAGE;
  const costBasis = (amount / position.total_tokens) * position.total_sol_spent;
  const pnl = solReceived - costBasis;

  // Update balance
  db.prepare('UPDATE accounts SET paper_balance = paper_balance + ?, total_trades = total_trades + 1').run(solReceived, pnl, wallet);

  // Update stats
  if (pnl > 0) {
    db.prepare('UPDATE accounts SET winning_trades = winning_trades + 1, current_streak = current_streak + 1').run(pnl, wallet);
  } else {
    db.prepare('UPDATE accounts SET current_streak = 0, worst_trade = MIN(worst_trade, ?)').run(pnl, wallet);
  }
}

```

```

}

// Update or close position
const remainingTokens = position.total_tokens - amount;
if (remainingTokens <= 0.000001) {
  db.prepare('DELETE FROM positions WHERE wallet = ? AND token_mint = ?').r
} else {
  const remainingSol = position.total_sol_spent - costBasis;
  db.prepare('UPDATE positions SET total_tokens = ?, total_sol_spent = ? WH
    .run(remainingTokens, remainingSol, wallet, tokenMint);
}

// Log trade
db.prepare('INSERT INTO trades (wallet, token_mint, token_name, token_symbo
  .run(wallet, tokenMint, tokenName, tokenSymbol, 'sell', solReceived, amou

// Track participation
const today = new Date().toISOString().split('T')[0];
db.prepare('INSERT INTO participation_metrics (wallet, date, trades_today,
  .run(wallet, today);
}

// Return updated state
return {
  account: db.prepare('SELECT * FROM accounts WHERE wallet = ?').get(wallet),
  positions: db.prepare('SELECT * FROM positions WHERE wallet = ?').all(walle
};
}

```

ANTI-GAMING RULES

- Max 20 open positions at once
- Min 0.1 SOL per trade
- Max 5 participation points per day (trades still execute, just no extra points)
- Balance reset: if paper_balance < 1 SOL and no open positions, allow reset to 100 SOL (max once per 7 days, flag the account)

MOBILE RESPONSIVENESS (CRITICAL)

Must work perfectly on 375px-430px width:

- Sidebar → bottom tab bar

- Chart: 250px height, full width
 - Trade panel: full width, stacked below chart
 - Buttons: full width, 48px min height
 - Tables → card layouts on mobile
 - Touch targets: 44px minimum
 - No horizontal scroll
 - All text readable without zoom
-

ERROR HANDLING

- API down: show cached data + subtle "updating..." indicator. NEVER show errors to users.
 - Price unavailable: disable trade button, show "Price unavailable" in muted text
 - WebSocket disconnect: auto-reconnect, show subtle reconnecting indicator
 - Trade fails: toast "Trade failed — try again", no balance change
 - Empty states: helpful messages ("Search for a token to start trading")
 - Never show: raw errors, stack traces, undefined, NaN, null, broken layouts
 - All numbers: proper formatting (commas, max 6 decimal places for small prices)
-

THINGS TO NOT INCLUDE

- No contract address bar
 - No token/project references
 - No airdrop UI or reward calculations
 - No "About Us" page with corporate text
 - No PFP generator
 - No mention of fund management, distributions, or tokenomics
 - No "Coming Soon" text on Phase 1 pages — those must be fully functional
 - No emojis in the UI
 - No meme language
-

ENVIRONMENT VARIABLES (Replit Secrets)

Plain Text

```
HELIUS_API_KEY=<already set>  
VITE_HELIUS_RPC_URL=https://mainnet.helius-rpc.com/?api-key=<key>
```

No key needed for: PumpPortal WebSocket, Jupiter Price API, DexScreener API.

SUMMARY OF WHAT TO BUILD NOW (Phase 1 only)

Fully functional:

1. ☒ Wallet connection (Phantom + Solflare)
2. ☒ Trading Desk with live charts, trade execution, positions, history, watchlist
3. ☒ Markets page with trending, new launches, migrations, gainers
4. ☒ Portfolio page with stats, performance chart, full history
5. ☒ PumpPortal WebSocket for live PumpFun data
6. ☒ All database schemas (including Phase 2/3 tables)
7. ☒ Analytics data collection (views, searches, trades)

Placeholder only:

- Leaderboard page → "Coming Soon"
- Utilities page → "Coming Soon"

DO NOT BUILD YET:

- Leaderboard calculations or ranking logic
- Competition systems
- Reward/airdrop calculations
- Wallet cleanup tools
- Any token-related features